

Reviewer Pack — Minimal Order of Diophantine Equivalence Classes and SAT Cla...

Entry cb51999a8ddc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 09:08:10 UTC

Minimal Order of Diophantine Equivalence Classes and SAT Clause Subset Complexity

Entry ID: cb51999a8ddc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 09:08:10 UTC

1. Conjecture

Field A (mathematical branch): Diophantine Geometry **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subset Complexity)

Statement:

For every Boolean satisfiability problem with n clauses, the minimal order of equivalence classes in its associated diophantine system over a finite field is linearly correlated with its SAT clause subset complexity, such that $\text{MinimalOrder}(\varphi) = \Theta(\text{SATComplexity}(\varphi))$ for any Tseitin formula φ_G .

Rationale (proposer's reasoning):

Diophantine systems provide a way to encode Boolean logic into arithmetic problems, which could potentially reveal deeper connections between arithmetic properties and complexity measures. The minimal order of equivalence classes in the diophantine system may capture structural information about the satisfiability problem that is not immediately evident from its clause subset complexity.

Taxonomy category: DiophantineGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 24582416d5e99e3d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For Tseitin formulas with n clauses, if $\text{MinimalOrder}(\varphi) / \text{SATComplexity}(\varphi)$ is within $\pm 10\%$ of 1 for all 30 seeds and all φ_G , then support is provided; otherwise, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_formula(n):
        literals = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for i in range(1, n+1):
            clauses.append([f'x{i}'])
        for i in range(1, n+1):
            for j in range(i+1, n+1):
                clauses.append([f'-x{i}', f'-x{j}', f'x{i+j-1}'])
        return literals, clauses

    def diophantine_system(literals, clauses):
        n = len(literals)
        A = [[0] * (n + 1) for _ in range(n)]
        b = [0] * n
        for clause in clauses:
            if len(clause) == 1:
                i = int(clause[0][1:]) - 1
                A[i][i] = 1
                b[i] = 1 if clause[0][0] != '-' else -1
            elif len(clause) == 2:
                i, j = [int(l[1:]) - 1 for l in clause if l[0] != '-']
                A[i][j] = 1
                A[j][i] = 1
        return A, b

    def gaussian_elimination(A, b):
        n = len(b)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
```

```

        max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            A[j][i:] = [A[j][k] - factor * A[i][k] for k in range(i, n+1)]
            b[j] -= factor * b[i]
    return A, b

def minimal_order(A):
    n = len(b)
    rank = 0
    for row in A:
        if any(row):
            rank += 1
    return rank

def sat_clause_subset_complexity(clauses):
    return len(clauses)

results = []
for n in [5, 10, 15, 20, 30, 40]:
    literals, clauses = tseitin_formula(n)
    A, b = diophantine_system(literals, clauses)
    _, _ = gaussian_elimination(A, b)
    minimal_order_value = minimal_order(A)
    sat_complexity = sat_clause_subset_complexity(clauses)
    results.append({
        "n": n,
        "minimal_order": minimal_order_value,
        "sat_complexity": sat_complexity
    })

mean_order = sum(r["minimal_order"] for r in results) / len(results)
mean_complexity = sum(r["sat_complexity"] for r in results) / len(results)
ratio = mean_order / mean_complexity

conjecture_holds = abs(ratio - 1) <= 0.1
counterexample = "" if conjecture_holds else f"Ratio {ratio} not within ±10% of 1"

return {
    "metric_name": "MinimalOrder/SATComplexityRatio",
    "metric_value": ratio,
    "instances_tested": len(results),
    "n_max": max(r["n"] for r in results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

```

```

results.append(trial_result)

mean_ratio = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Ratio out of ±10%\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

72106825 not within ±10% of 1'}
TRIAL: {'metric_name': 'MinimalOrder/SATComplexityRatio', 'metric_value': 0.0712166172106825, 'inst': 72106825}
TRIAL: {'metric_name': 'MinimalOrder/SATComplexityRatio', 'metric_value': 0.0712166172106825, 'inst': 72106825}
TRIAL: {'metric_name': 'MinimalOrder/SATComplexityRatio', 'metric_value': 0.0712166172106825, 'inst': 72106825}
TRIAL: {'metric_name': 'MinimalOrder/SATComplexityRatio', 'metric_value': 0.0712166172106825, 'inst': 72106825}
TRIAL: {'metric_name': 'MinimalOrder/SATComplexityRatio', 'metric_value': 0.0712166172106825, 'inst': 72106825}
TRIAL: {'metric_name': 'MinimalOrder/SATComplexityRatio', 'metric_value': 0.0712166172106825, 'inst': 72106825}
TRIAL: {'metric_name': 'MinimalOrder/SATComplexityRatio', 'metric_value': 0.0712166172106825, 'inst': 72106825}
TRIAL: {'metric_name': 'MinimalOrder/SATComplexityRatio', 'metric_value': 0.0712166172106825, 'inst': 72106825}
TRIAL: {'metric_name': 'MinimalOrder/SATComplexityRatio', 'metric_value': 0.0712166172106825, 'inst': 72106825}

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considers a small range of n (up to 40), which may not be sufficient to establish the linear correlation between minimal order and SAT clause subset complexity. The conjecture's validity might depend on properties that are only apparent for larger instances.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that for at least one seed, the ratio of MinimalOrder to SATComplexity is outside the $\pm 10\%$ range of 1, which contradicts the | next: Further investigation is needed to determine if the conjecture holds for larger instances or under different conditions.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13374
2	preregistration	ollama_remote	glm4:latest	0	0	9488
3	novelty	ollama_remote	glm4:latest	0	0	8521
4	novelty	ollama_remote	glm4:latest	0	0	8520
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16997
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14607
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9688
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12862
9	critic	ollama_remote	glm4:latest	0	0	18761
10	judge	ollama_remote	glm4:latest	0	0	9001

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 121818 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/cb51999a8ddc.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/cb51999a8ddc.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/cb51999a8ddc.tar.gz* (if generated)